Unit 1

Introduction to Python

- The basic elements of Python
- Branching programs
- Strings and Input
- Iteration
- Functions and Scoping, Specifications, Recursion
- Global variables, Modules, Files
- Tuples, Lists and Mutability
- Functions as Objects, Strings
- Tuples and Lists, Dictionaries

What is PYTHON ?

- Python is an interpreter, object-oriented, high-level programming and scripting language.
- It was created by “Guido van Rossum” and first released in 1991. Python can be used for developing desktop GUI applications, websites and web applications.
- Python can also be translated into binary code like java. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance.
- Python supports modules and packages, which encourages program modularity and code reuse.
- The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms, and can be freely distributed.

- Since there is no compilation step, the edit-test-debug cycle is incredibly fast.
- Debugging Python programs is easy: a bug or bad input will never cause a segmentation fault.
- Instead, when the interpreter discovers an error, it raises an exception.
- When the program doesn't catch the exception, the interpreter prints a stack trace.
- A source level debugger allows inspection of local and global variables, evaluation of arbitrary expressions, setting breakpoints, stepping through the code a line at a time, and so on.
- The debugger is written in Python itself, testifying to Python's introspective power.
- On the other hand, often the quickest way to debug a program is to add a few print statements to the source: the fast edit-test-debug cycle makes this simple approach very effective.

The basic elements of Python

- Python programs can be written using any text editor and should have the extension .py.
- Python programs do not have a required first or last line, but can be given the location of python as their first line: #!/usr/bin/python and become executable.
- Otherwise, python programs can be run from a command prompt by typing python file.py.
- There are no braces {} or semicolons ; in python.
- It is a very high level language. Instead of braces, blocks are identified by having the same indentation.

Data Types and Variable :

- Data types classify the kind of value a variable can hold. Python has several built-in data types

- **Numeric Types**:
- **int:**
  - Represents whole numbers (e.g., 10, -5, 0).
- **float:**
  - Represents decimal numbers (e.g., 3.14, -2.5).
- **complex:**
  - Represents complex numbers (e.g., 2+3j).

- **String Type:**
  - **str:**
    - Represents sequences of characters, enclosed in single or double quotes (e.g., "hello", 'world').
- **Sequence Types:**
  - list:
    - Represents ordered, mutable collections of items, enclosed in square brackets (e.g., [1, 2, "apple"]).
  - tuple:
    - Represents ordered, immutable collections of items, enclosed in parentheses (e.g., (1, 2, "apple")).
  - range:
    - Represents a sequence of numbers.

- Mapping Type:
  - dict:
    - Represents collections of key-value pairs, enclosed in curly braces (e.g., {"name": "John", "age": 30}).

- Set Types:
  - set:
    - Represents unordered collections of unique items, enclosed in curly braces (e.g., {1, 2, 3}).
  - frozenset:
    - Represents immutable sets.

- Boolean Type:
  - bool:
    - Represents logical values, either True or False.

Variables

- Variables are names that refer to values stored in memory. In Python, variables:

- Do not require explicit type declaration. The type is inferred from the assigned value.

- Can change their data type during the program's execution due to Python's dynamic typing.
- Are case-sensitive (e.g., age and Age are different variables).
- Must start with a letter or underscore, can contain alphanumeric characters and underscores, and cannot start with a number.

- Variables are assigned values using the = operator. For example:
- x = 10 # x is an integer
- y = "hello" # y is a string
- z = [1, 2, 3] # z is a list

- Type Checking
- print(type(x)) # Output: <class 'int'>
- print(type(y)) # Output: <class 'str'>
- print(type(z)) # Output: <class 'list'>

Python Strings

- Strings in python are surrounded by either single quotation marks, or double quotation marks.

- 'hello' is the same as "hello".

- You can display a string literal with the print() function:

- print("Hello")
  print('Hello')

Assign String to a Variable

- a = "Hello"
  print(a)

- **Slicing**
- You can return a range of characters by using the slice syntax.
- Specify the start index and the end index, separated by a colon, to return a part of the string.
- b = "Hello, World!"
  print(b[2:5])

- **Modify Strings**
- Upper Case
- a = "Hello, World!"
  print(a.upper())
- Lower Case
- a = "Hello, World!"
  print(a.lower())
- Remove Whitespace
- a = " Hello, World! "
  print(a.strip()) # returns "Hello, World!“
- Replace String
- a = "Hello, World!"
  print(a.replace("H", "J"))

- **String Concatenation**
- a = "Hello"
  b = "World"
  c = a + b
  print(c)
- **String Format**
- F-Strings
- F-String was introduced in Python 3.6, and is now the preferred way of formatting strings.
- To specify a string as an f-string, simply put an f in front of the string literal, and add curly brackets {} as placeholders for variables and other operations.
- age = 36
  txt = f"My name is John, I am {age}"
  print(txt)

Escape Character

- To insert characters that are illegal in a string, use an escape character.
- An escape character is a backslash \ followed by the character you want to insert.
- An example of an illegal character is a double quote inside a string that is surrounded by double quotes:
- txt = "We are the so-called \"Vikings\" from the north.“
- \' Single Quote
- \\ Backslash
- \n New Line
- \t Tab
- \b Backspace
- \f Form Feed
- \ooo Octal value
- \xhh Hex value

Python If ... Else(Branching programs)

- Python supports the usual logical conditions from mathematics:

- Equals: a == b
- Not Equals: a != b
- Less than: a < b
- Less than or equal to: a <= b
- Greater than: a > b
- Greater than or equal to: a >= b

If statement:

- a = 33
  b = 200
  if b > a:
    print("b is greater than a")

- Elif
- The elif keyword is Python's way of saying "if the previous conditions were not true, then try this condition".
- ```
  a = 33
  b = 33
  if b > a:
    print("b is greater than a")
  elif a == b:
    print("a and b are equal")
  ```
  ```

•Else

- The else keyword catches anything which isn't caught by the preceding conditions.
- a = 200
  b = 33
  if b > a:
    print("b is greater than a")
  elif a == b:
    print("a and b are equal")
  else:
    print("a is greater than b")

- Short Hand If
- if a > b: print("a is greater than b")

- Short Hand If ... Else
- a = 2
  b = 330
  print("A") if a > b else print("B")

Nested If

- x = 41

```
if x > 10:
  print("Above ten,")
  if x > 20:
    print("and also above 20!")
  else:
    print("but not above 20.")
```

And Keyword

- The and keyword is a logical operator, and is used to combine conditional statements:
- a = 200
  b = 33
  c = 500
  if a > b and c > a:
    print("Both conditions are True")

Or Keyword

- The or keyword is a logical operator, and is used to combine conditional statements:
- a = 200
  b = 33
  c = 500
  if a > b or a > c:
    print("At least one of the conditions is True")

Not Keyword

- The not keyword is a logical operator, and is used to reverse the result of the conditional statement:
- a = 33
  b = 200
  if not a > b:
    print("a is NOT greater than b")

Python Match

- Instead of writing many if..else statements, you can use the match statement.
- The match statement selects one of many code blocks to be executed.
- Syntax
- match *expression*:

```
  case x:
    code block
  case y:
    code block
  case z:
    code block
```

Example

- ```
  day = 4
  match day:
    case 1:
      print("Monday")
    case 2:
      print("Tuesday")
    case 3:
      print("Wednesday")
    case 4:
      print("Thursday")
    case 5:
      print("Friday")
    case 6:
      print("Saturday")
    case _:
      print("Sunday")
  ```
  ```

Python Loops(Iteration)

- In Python, loops are control flow structures that allow you to execute a block of code repeatedly.

- Python has two primitive loop commands:
- while loops
- for loops

The while Loop

- With the while loop we can execute a set of statements as long as a condition is true.
-
```
i = 1
while i < 6:
  print(i)
  i += 1
```

- The break Statement
- With the break statement we can stop the loop even if the while condition is true:
- ```
  i = 1
  while i < 6:
    print(i)
    if i == 3:
      break
    i += 1
  ```
- The continue Statement
- With the continue statement we can stop the current iteration, and continue with the next:
- ```
  i = 0
  while i < 6:
    i += 1
    if i == 3:
      continue
    print(i)
  ```

Python For Loops

- A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-orientated programming languages.
- With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.
- fruits = ["apple", "banana", "cherry"]
  for x in fruits:
    print(x)

- for x in range(6):
- print(x)
- Output: 0 1 2 3 4 5

- for x in range(3, 8):
- print(x)
- Output:

3

4

5

6

7

Function and Scoping

- Function:
- A function is a block of organized, reusable code that is used to perform a single, related action.
- Functions provide better modularity for your application and a high degree of code reusing.

How to define a function :-

- Function blocks begin with the keyword def followed by the function name and parentheses ().
- Any input parameters or arguments should be placed within these parentheses.
- The code block within every function starts with colon : and is indented.
- The statement return [expression] exits a function, optionally passing back an expression to the caller.

Syntax :-

def functionname (parameters) :

statement / “function_docstring”

return [expression]

Example :-

```
#define a function
def printMe(str) :
#this prints a passed string into this function
        print (str)
        return
#calling function
printMe(“Hello Students”)
```

Output :-

Hello students

Scope:

- Variable can only reach the area in which they are defined, which is called scope.
- Think of it as the area of code where variable can be used.
- Python supports global variable (usable in the entire program) and local variables.
- By default, all function, it’s required to explicitly define ‘global variable’.

```
myStr = "This is global variable"
a=10
def myVar() :
    myStr = "this is local variable"
    print(a)
    print(myStr)
myVar()
print(myStr)
```

Recursion

- Recursion means that the function will continue to call itself and repeat its behavior until some condition is met to return a result.
- All recursive functions share a common structure made up of two parts: base case and recursive case

```
def factorial_recursive(n) :
#base case : 1! = 1
    if n == 1 :
        print (n, end = '=')
        return 1
#recursive case : n! = n*(n-1)!
    else:
        print(n, end = 'X')
    return n*factorial_recursive(n-1)
i = input('Enter Any Number : ')
print (factorial_recursive(int(i)))
```

Python Modules

- Modules refer to a file containing python statements and definitions.
- A file containing python code for eg. “abc.py” is called a module name is “abc”.
- Python uses module to break down large programs into small manageable and organized file.
- Simply a module is a file consisting of python code.
- A module can define function, classes and variables Let’s first create a module named “myAddMod.py” on path

- mymodual.py
- def myAdd(fVal, sVal) :
- return (fVal + sVal)

- testfile.py
- import mymodual as add
- print(add.myAdd(50,6))

Python files/files handling

- File is a named location on disk to store related information.
- It is used to permanently store data in a non-volatile memory (eg :- hard disk).
- RAM is volatile which loses its data when computer is turned off.

How to open a file ?

- Python has a built-in function open() to open a file.
- >>>f = open(“test.txt”) #open file in current directory
- >>>f = open(“C:\\Python\\README.txt”) #specifying full path
- We specify the mode while opening a file.
- The default is reading in the text mode.

Mode	Description
'r'	Opening file for reading. (default)
'w'	Open a file writing. Creating a new file if it does not exist or truncates the file if it exists.
'x'	Open a file for exclusive creation. If the file already exist, the operation fails
'a'	Open for appending at the end of the file without truncating it. Creates a new file if it does not exist.
't'	Open in text mode. (default)
'b'	Open binary mode.
'+'	Open a file for updating (reading and writing)

- with open ('C:\\Users\\This Pc\\OneDrive\\Desktop\\Bca\\Python\\Python\\test.txt', 'w', encoding = 'utf-8') as f:

  f.write('Hello\n')

  f.write('czmg\n')

  f.write('Students\n')

  f.close()

Lists and Mutability

- Mutable means Changeable.
- Everything in Python is an object.
- You have to understand that Python represents all its data as objects.
- An object’s mutability is determined by its type.
- Some of these objects like lists and dictionaries are mutable, meaning you can change their content without changing their identity.
- Other objects like integers, floats, strings and tuples are objects that cannot be changed.
- Strings are immutable in Python, which means you cannot change an existing string.

- myList = [10, 20, 30, 40, 50]
- print(myList)
- myList[0] = 5
- print(myList)
- Output :- [10, 20, 30, 40, 50] [5, 20, 30, 40, 50]

myTuple = (10, 20, 30, 40, 50)

- print(myTuple)
- myList[0] = 5
- print(myTuple)
- output
- Traceback (most recent call last): File "C:\Test.py", line 3, in <module> myTuple[0] = 5 TypeError: 'tuple' object does not support item assignmen

Function as object

- In Python, functions are considered "first-class objects," meaning they are treated like any other object, such as integers, strings, or lists. This concept allows for powerful and flexible programming paradigms.
- Key implications of functions as first-class objects:
- **Assigned to variables:** Functions can be assigned to variables, and then called using the variable name.
-

```
def greet(name):
    return f"Hello, {name}!"
my_function = greet
print(my_function(“Ty Students")) # Output: Hello, Alice!
```

Passed as arguments to other functions:

Functions can be passed as arguments to other functions, enabling higher-order functions like map, filter, or custom functions that take other functions as input.

```
def apply_operation(addfunc, x, y):
    return addfunc(x, y)

def add(a, b):
    return a + b

result = apply_operation(add, 5, 3)
print(result) # Output: 8
```

Difference between Tuples and Lists, Dictionaries

List	Tuples	Dictionary
A list is mutable	A tuple is immutable	A dictionary is mutable
Lists are dynamic	Tuples are fixed size in nature	In values can be of any data type and can repeat, keys must be of immutable type
List are enclosed in brackets[ ] and their elements and size can be changed	Tuples are enclosed in parenthesis ( ) and cannot be updated	Tuples are enclosed in curly braces { } and consist of key:value
Homogenous	Heterogeneous	Homogenous
Example: List = [10, 12, 15]	Example: Words = ("spam", "egss") Or Words = "spam", "eggs"	Example: Dict = {"ram": 26, "abi": 24}
Access: print(list[0])	Access: print(words[0])	Access: print(dict["ram"])
Can contain duplicate elements	Can contain duplicate elements. Faster compared to lists	Cant contain duplicate keys, but can contain duplicate values
Slicing can be done	Slicing can be done	Slicing can't be done
Usage: ❖ List is used if a collection of data that doesnt need random access. ❖ List is used when data can be modified frequently	Usage: ❖ Tuple can be used when data cannot be changed. ❖ A tuple is used in combination with a dictionary i.e.a tuple might represent a key.	Usage: ❖ Dictionary is used when a logical association between key:value pair. ❖ When in need of fast lookup for data, based on a custom key. ❖ Dictionary is used when data is being constantly modified.